BITS Pilani, Pilani Campus

CS F213 Object Oriented Programming

Summer Term 2026

High-Concurrency Vendor Order Management System (Vendor Engine)

Track 4: Application Development with Java Frameworks

Prepared by:
Group 25
Rohit Kumar Patel
Student ID: 2023B5A30814P

Submitted to:
Prof. Aneesh Chivukula

July 11, 2026

Contents

1 Introduction & Operational Context **4**
1.1 Operational Context . . . 4
1.2 The Bottleneck Challenge . . . 4
1.3 Proposed Solution: The Vendor Engine . . . 4

2 Problem Definition **5**
2.1 Inputs to the System . . . 5
2.2 Outputs of the System . . . 5
2.3 System Success Metrics (Design Targets) . . . 6

3 Scope & Feasibility **6**
3.1 Project Scope Definition . . . 6
3.2 Feasibility Analysis . . . 7
3.3 3-Week Implementation Timeline . . . 7

4 System Architecture **7**
4.1 Architectural Layers . . . 7
4.2 Layered Diagram (TikZ) . . . 8
4.3 Architectural Data Flow . . . 8

5 UML Class Diagram & OOP Principles **9**
5.1 UML Class Diagram . . . 9
5.2 OOP Design Principles Applied . . . 9
5.2.1 Encapsulation (State Shielding) . . . 9
5.2.2 Abstraction (Decoupling Interface from Implementation) . . . 10
5.2.3 Inheritance & Polymorphism . . . 10
5.2.4 Composition over Inheritance . . . 10

6 Design Patterns **10**
6.1 Design Patterns Mapping Table . . . 10
6.2 Implementation Verification & Details . . . 11
6.2.1 Model-View-Controller (MVC) . . . 11
6.2.2 Observer Pattern . . . 11
6.2.3 Producer-Consumer Pattern . . . 12
6.2.4 Singleton Pattern . . . 12
6.2.5 Strategy Pattern . . . 12

7 Concurrency Model & No-Deadlock Argument **12**
7.1 Concurrency and Threading Infrastructure . . . 13
7.2 The Structural No-Deadlock Argument . . . 13
7.3 Memory Visibility Constraints . . . 14
7.4 Daemon Thread Selection . . . 14

8 Advanced Java Features **14**
8.1 Advanced Java Features Mapping Table . . . 14
8.2 Key Implementation Details . . . 15
8.2.1 Serialization of Custom Objects . . . 15
8.2.2 Static vs. Non-Static Inner Classes . . . 15

9 Offline Resilience & Network Failover **16**
9.1 Network State Detection . . . 16
9.2 Failover and Recovery Flow . . . 16
9.3 Detailed State-Transition Walkthrough . . . 17
9.3.1 Transition to Offline Mode . . . 17
9.3.2 Transition to Online Mode (Reconnection & Sync) . . . 17
9.4 Testing and Demonstration Interface . . . 17

10 GUI / View Layer Design **17**
10.1 Multi-Role Navigation Flow . . . 18
10.2 Signature Custom Visual Components . . . 18
10.2.1 Docket Ring Component (`DocketRingComponent`) . . . 18
10.2.2 Ticket Card Border (`TicketCardBorder`) . . . 18
10.2.3 Themed Dialog (`ConfirmLogoutDialog`) . . . 19
10.3 View Implementations . . . 19
10.3.1 Kitchen Dashboard (`KitchenView`) . . . 19
10.3.2 Merchant Operations Center (`AdminView`) . . . 19
10.4 Event Dispatch Thread (EDT) Discipline . . . 19

11 Integration with the Festival Platform **20**
11.1 Platform Integration Interfaces . . . 20
11.1.1 Static Configuration Ingestion (`stalls.json`) . . . 20
11.1.2 Simulated Real-Time Ingest Stream . . . 20
11.1.3 Reconnection Synchronization Payload (`sync_payload.json`) . . . 20
11.2 Local Persistence Architecture (`offline_stalls.ser`) . . . 21
11.3 JDBC-Ready DAO Design . . . 21

12 Testing & Verification **21**
12.1 JUnit 5 Test Suite Description . . . 21
12.2 Verification of Automated Test Run . . . 22
12.3 Concurrency Testing Strategy . . . 22

13 Results & Success Metrics **23**
13.1 Metrics Comparison Table . . . 23
13.2 Discussion of Performance Results . . . 24
13.2.1 Data Integrity and Preservation . . . 24
13.2.2 Priority Accuracy . . . 25

14 Innovation & Additional Enhancements **25**
14.1 Glanceable Kitchen UX: The Docket Ring . . . 25
14.2 Contextual Kanban Workspace Layout . . . 25
14.3 Tactile Theme Styling: Ticket Card Border . . . 26
14.4 Administrative Bottleneck Diagnostics . . . 26

15 Limitations & Future Work **26**
15.1 Current Limitations . . . 26
15.2 Future Work & Architecture Enhancements . . . 26

16 Conclusion **27**

A Appendix A: Build & Run Instructions **28**
A.1 System Prerequisites . . . 28
A.2 Standard Build and Test Lifecycle Commands . . . 28
A.3 Running the Application . . . 28
A.4 Simulating Network Failover During Live Demos . . . 28

B Appendix B: Selected Code Listings **29**
B.1 Prioritization Strategy: OrderComparator.java . . . 29
B.2 Thread Pool Manager: ExecutorServiceManager.java . . . 30

C Appendix C: Test Suite Summary Table **31**
C.1 Test Summary Table . . . 31
C.2 Testing Execution Details . . . 32

1 Introduction & Operational Context

This report describes the design and implementation of the **High-Concurrency Vendor Engine**, a software component developed as part of the CS F213 Object Oriented Programming course project at BITS Pilani (Summer Term 2026). The project forms part of Track 4 (Application Development with Java Frameworks), focusing on building robust, high-performance, and resilient digital solutions to address real-world operational bottlenecks.

1.1 Operational Context

BITS Pilani's annual cultural festival is a large-scale event serving a user base of 5,000–6,000 active participants. The digital ecosystem supporting the festival facilitates real-time operations, including:

- Ticketing and entry validation.
- Event scheduling and coordination across 90+ events.
- Food ordering at multiple vendor stalls.
- Digital wallet transactions totaling over ₹1.5+ crore in value.
- Live dashboards displaying aggregate sales, event progress, and crowd distribution.

The central backend of this ecosystem is implemented as a Django monolith, supported by PostgreSQL for persistent storage, Redis as an in-memory cache, Celery for asynchronous background task execution, and WebSockets for real-time notifications. The primary user interface for participants is a mobile application.

1.2 The Bottleneck Challenge

During peak festival hours, food stalls experience an intense order overload. In this high-throughput scenario, the central Django monolith and its WebSocket handlers frequently encounter performance degradation. The major operational pain points identified include:

1. **WebSocket Connection Drops**: Simultaneous order streams from thousands of users strain the central WebSocket server, leading to connection dropouts and order delivery delays.
2. **Stall App Lag**: The vendor-facing application lags due to synchronous network dependency, causing long waiting lines, order processing delays, and incorrect revenue reporting.
3. **Lack of Offline Resilience**: In the event of a Wi-Fi or network failure on the festival floor, vendors lose the ability to accept, process, or track orders. This leads to substantial data loss, double-booking, and financial discrepancies.

1.3 Proposed Solution: The Vendor Engine

To resolve these bottlenecks, the **High-Concurrency Vendor Engine** has been developed. It is a lightweight, local desktop application built in Java 17 that runs on-premise at each vendor stall. By moving order routing, prioritization, and queue management to local vendor machines, the system eliminates direct WebSocket dependencies during peak operation.

Key features of the Vendor Engine include:

- **High-Concurrency Ingestion**: An 8-thread consumer pool capable of parsing and routing up to 50 simulated orders/second.

- **Strict Prioritization**: A custom queue sorting strategy that prioritizes VIP festival-goers and handles tie-breakers chronologically (First-In, First-Out).
- **Autonomous Offline Failover**: An independent network monitoring daemon that detects connectivity drops and serializes the system state locally to prevent data loss.
- **Automatic Synchronization**: On reconnection, the daemon automatically compiles and uploads served orders back to the central monolith, restoring consistency.

2 Problem Definition

To construct a high-performance local dashboard for food stall operators, the problem must be formally defined in terms of its operational bounds, input schemas, output requirements, and success metrics. The Vendor Engine functions as a local, concurrent mediator that decouples the network-dependent order transmission from real-time kitchen operations.

2.1 Inputs to the System

The application ingests several distinct inputs to manage stall configurations and order flows:

- **Stalls Configuration File (`stalls.json`)**: A static JSON configuration file loaded at application startup defining registered food stalls. It maps unique stall IDs to human-readable names.
- **Simulated WebSocket Order Stream**: A high-throughput stream of JSON order payloads generated by the simulation layer. Each order includes:
  - `orderId`: A unique string representation of a UUID.
  - `stallId`: The ID of the target stall.
  - `priority`: The priority tier of the order (`VIP` or `STANDARD`).
  - `createdAt`: A Unix millisecond timestamp representing when the order was placed.
  - `items`: A JSON array of line items, where each item contains `itemName`, `quantity`, and `unitPrice`.
- **Heartbeat Connectivity Signal (`network.flag`)**: A control file polled periodically by the system's background network monitor. The file contains a single string, either `"up"` or `"down"`, simulating WebSocket network connectivity.
- **User Authentication Credentials**: User-selected roles (`KITCHEN_WORKER` or `MERCHANT_ADMIN`) and assigned stall IDs supplied via the graphical login screen.

2.2 Outputs of the System

The application generates several visible and persistent outputs:

- **Live Prioritized Kanban Boards**: Java Swing-based graphical displays showing sorted active orders in three status-based columns (Pending, Accepted, Ready to Serve).
- **Offline Binary State Backup (`offline_stalls.ser`)**: A serialized binary snapshot of the entire application state (including order queues and configurations) written to disk automatically when network loss is detected.
- **Reconnection Synchronization Payload (`sync_payload.json`)**: A structured JSON file containing all orders processed and served locally during an offline period, written upon network restoration to sync with the backend.

- **Real-Time Operational Analytics**: Dynamic visual metrics displayed to administrators, including total revenue in Indian Rupees (₹), total active queue depths, and live revenue trend charts.

2.3 System Success Metrics (Design Targets)

The system is architected to satisfy the following operational benchmarks under peak-hour conditions. Per the project specifications, these are designated as primary **design targets**:

Table 1: System Performance and Correctness Design Targets

Metric Class	Operational Success Target	Design Justification
Data Integrity	Zero data loss during connection drops	Local memory caching + immediate serialization
UI Latency	$<$100ms UI refresh latency under load	Background threads + EDT separation (invokeLater)
Prioritization	$\geq$95% VIP ranking accuracy	Composite comparator sorting VIP over Standard
Failover Speed	$<$3s recovery and synchronization time	Quick parsing + batch JSON writing on reconnect
Concurrency	8-thread concurrent consumer processing	Fixed thread pool size structured for zero deadlocks

3 Scope & Feasibility

A critical aspect of software engineering is ensuring the project's scope is aligned with available timelines and resources. For the CS F213 OOP project, the implementation was constrained to a 3-week development cycle during the summer term. The scope and feasibility analysis below justifies the architectural choices and timeline partitioning.

3.1 Project Scope Definition

The system was scoped specifically to create a standalone desktop application rather than a full-scale web service, making it highly feasible for a single student developer while maintaining production-level complexity:

- **In-Scope Features**:
  - A local order management daemon that simulates real-time client traffic.
  - Multithreaded order consumption and local priority queue routing.
  - Graphical user interfaces for kitchen staff and merchant administrators.
  - local file-based offline serialization and batch synchronization.
  - A JUnit 5 unit testing suite achieving $\geq$80% code coverage on core business logic.
- **Out-of-Scope Features**:
  - A production-grade network layer (replaced by file-based heartbeat simulation).
  - A physical SQL database integration (the application uses in-memory data structures, designed to be "JDBC-ready" for future DAO expansion).
  - Automated graphical user interface unit testing.

3.2 Feasibility Analysis

The project's feasibility is supported by three primary architectural decisions:

1. **Zero-Dependency Core**: By avoiding heavy external frameworks (such as Spring Boot or JavaFX) and third-party utility libraries (such as Jackson or Apache Commons), compilation is fast, and runtime errors are minimized. The custom-written `JsonUtil` recursive-descent parser keeps the project dependency-free (requiring only JUnit 5 for tests).
2. **Compact Class Hierarchy**: The entire codebase consists of 36 Java files (including views and tests), aligning with the feasibility target of approximately 20–25 core production classes.
3. **Standardized Build Toolchain**: Using Maven for build orchestration and compilation ensures that the application compiles into a single executable fat JAR, making deployment simple and platform-independent.

3.3 3-Week Implementation Timeline

The project was structured across a three-week schedule, separating milestones to prevent integration blockages:

Week 1: Core Foundation Development of the data models (`Order`, `Stall`, `AppState`), the concurrent Producer-Consumer pipeline, and the central controller routing logic. Core unit tests for comparisons and queues were written.

Week 2: GUI & Observer Wiring Design of the user interfaces (`LoginView`, `KitchenView`, `AdminView`) using Swing. Wiring the views as observers of the controller and utilizing `SwingUtilities.invokeLater()` to ensure thread-safety on the Event Dispatch Thread (EDT).

Week 3: Resilience, Testing & Polish Development of the `NetworkMonitorDaemon`, serialization backup (`offline_stalls.ser`), and synchronization client (`sync_payload.json`). Finalizing the JUnit 5 test suite to exceed coverage targets and recording the operational demonstration video.

4 System Architecture

The Vendor Engine is designed around a strictly decoupled, five-tier layered architecture that separates data structures, concurrency handling, business mutation logic, storage operations, and the user interface. This structural separation enforces a clean Model-View-Controller (MVC) relationship, ensuring that the model layer has zero knowledge of visual layouts or network states.

4.1 Architectural Layers

The system consists of five distinct layers:

1. **Simulation Layer**: Responsible for loading stall configuration files and generating synthetic peak-hour JSON streams.
2. **Concurrency Layer**: Implements the high-throughput Producer-Consumer pipeline using thread pools and thread-safe queues.
3. **Controller Layer**: Implements business logic, handles order status transitions, and maintains the registry of active observers.

4. **Model Layer**: Holds immutable-ish domain data structures with zero external dependencies.
5. **I/O and Storage Layer**: Executes offline binary serialization, polls network flags, and generates synchronization payloads.

4.2 Layered Diagram (TikZ)

Figure 1 visualizes the five layers, their component layout, and the data flow directions between them.

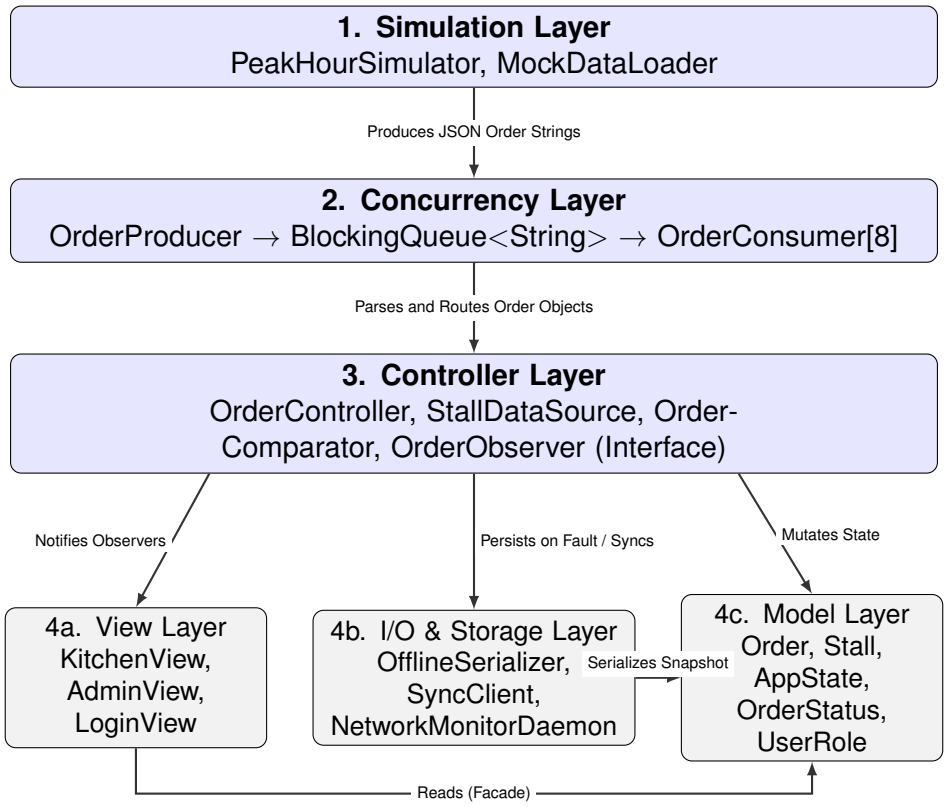

Figure 1: Layered Architecture and Data Flow Diagram

4.3 Architectural Data Flow

During runtime, order events flow through the system in a structured sequence:

1. **Ingestion**: The `PeakHourSimulator` generates an order payload as a flat JSON string and hands it to the `OrderProducer`. The producer pushes this string into a bounded `LinkedBlockingQueue` (capacity 1000). If the queue is full, backpressure is applied to the producer.
2. **Distribution**: Eight concurrent `OrderConsumer` threads pull JSON strings from the queue, parse them into `Order` objects using `JsonUtil`, and submit them to the `OrderController`.
3. **Routing**: The `OrderController` looks up the target `Stall` in its registry map. It enqueues the order into that stall's specific `PriorityBlockingQueue`, which sorts it using the `OrderComparator`.
4. **Notification**: The controller dispatches state updates on the Event Dispatch Thread (EDT) using `SwingUtilities.invokeLater()` to notify all registered observers (such as the GUI views).

5. **Failover**: If the `NetworkMonitorDaemon` detects a network drop, it grabs a deep snapshot of the `AppState` (stalls, queues, and metadata) and writes it as a binary serialized stream to `offline_stalls.ser` via the `OfflineSerializer`.

5 UML Class Diagram & OOP Principles

Object-Oriented Programming (OOP) forms the core architectural foundation of the Vendor Engine. By strictly adhering to OOP guidelines, the system achieves modularity, maintainability, and thread safety.

5.1 UML Class Diagram

Figure 2 shows the structural relations and class definitions for the core business logic, including field access types, interface implementations, and class associations.

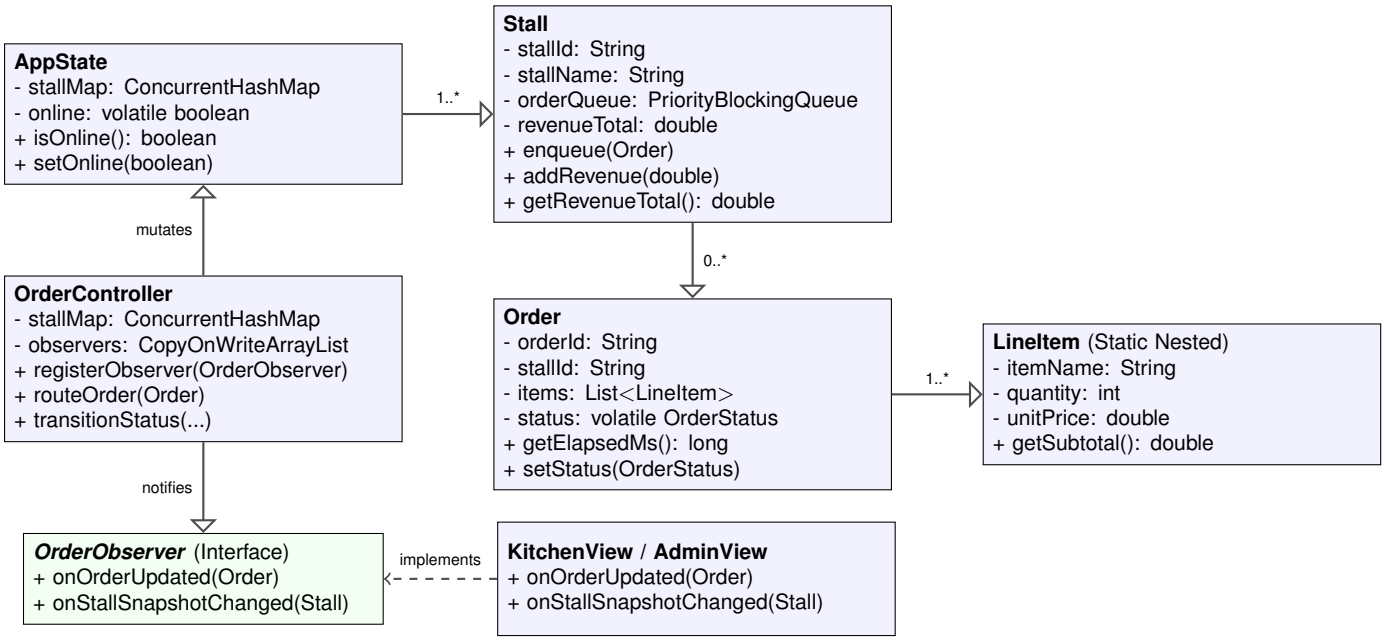

Figure 2: Core Component UML Class Diagram

5.2 OOP Design Principles Applied

The application applies four fundamental OOP principles to enforce modularity and prevent bugs:

5.2.1 Encapsulation (State Shielding)

Data integrity is maintained by guarding internal fields against unauthorized external mutations.

- All model variables (in `Order`, `Stall`, `LineItem`, and `AppState`) are marked as `private` and `final` where possible, allowing read-only access via public getter methods.
- The only mutable state in the domain layer is an order's status (`OrderStatus`). This is marked as `volatile` in `Order.java` and can only be altered via the controller's `transitionStatus()` method. No view is permitted to mutate status fields directly.
- The `StallDataSource` class implements a read-only facade pattern. It wraps the core stall map and exposes only unmodifiable views and summaries (e.g. sorted list of stall IDs, aggregate revenue totals) to prevent the `AdminView` from calling mutation routines.

5.2.2 Abstraction (Decoupling Interface from Implementation)

By separating architectural responsibilities, layers communicate via high-level contracts.

- The `OrderObserver` interface establishes an abstract notification boundary. The `OrderController` publishes updates to a generic list of observers without knowing if they represent a CLI log, a GUI table (`OrderTableModel`), or custom drawing panels.
- UI rendering details are abstracted away from model processing: classes under the `com.festival.vendorengine.model` package contain zero AWT or Swing imports, ensuring they can be reused in any environment.

5.2.3 Inheritance & Polymorphism

Polymorphic typing is extensively used to build robust classes and custom error flows:

- The system utilizes a hierarchical exception tree. Recoverable conditions extend checked exception classes (e.g. `StallNotFoundException` inherits from `Exception`, forcing the UI caller to handle it). Programmatic violations and parsing failures inherit from unchecked exceptions (e.g. `OrderProcessingException` extends `RuntimeException`).
- The custom GUI views extend Swing containers (`KitchenView` and `AdminView` inherit from `JFrame`; `ConfirmLogoutDialog` inherits from `JDialog`).

5.2.4 Composition over Inheritance

Rather than extending core data structures, components are built by composing standard structures.

- `Stall` encapsulates a `PriorityBlockingQueue<Order>` instead of extending it. This prevents exposing queue management APIs directly to outside classes and ensures thread safety is fully handled within the `Stall` boundary.

6 Design Patterns

To structure a complex, multi-threaded GUI application, recognized design patterns were utilized. These patterns ensure low coupling, high cohesion, and scalable expansion.

6.1 Design Patterns Mapping Table

Table 2 lists the implemented design patterns, their exact locations within the package structure, and their structural purpose.

Table 2: Design Pattern Implementation Mapping

Design Pattern	Target Files / Classes	Role in System	Structural Evidence & Rationale
Model-View-Controller	Packages: `model/`, `view/`, `controller/`	System partitioning	Decouples data, transitions, and visualization. Model package has zero Swing/AWT imports.
Observer	`OrderObserver`, `OrderController`	State change distribution	Decouples controller changes from table/view updates. Views register at startup and repaint on notify.
Producer-Consumer	`OrderProducer`, `OrderConsumer`, `BlockingQueue`	Ingestion rate decoupling	Prevents high simulated transaction rates from locking up database/file writing. Bounded queue handles overflow.
Singleton	`ExecutorServiceManager`	Resource pool control	Prevents thread-count explosions. Eager initialization ensures thread-safe, lock-free global access.
Strategy	`OrderComparator`	Priority ranking algorithm	Decouples order prioritizing logic from the queue. Different comparator strategies can be swapped at construction.
Facade / DAO	`StallDataSource`	Read-only data encapsulation	Exposes a read-only summary facade of stalls to the views, protecting mutating controller methods.

6.2 Implementation Verification & Details

6.2.1 Model-View-Controller (MVC)

The MVC layout is strictly enforced in the directory structure. To verify that no visual styling or Swing classes have leaked into the business/data logic, a grep command for Swing classes inside the model package yields zero matches:

```
grep -r "javax.swing" src/main/java/com/festival/vendorengine/model/
# Returns empty result (no UI leaks into domain model)
```

The model package holds pure data invariants, the controller coordinates mutations, and the view layer visualizes data.

6.2.2 Observer Pattern

The `OrderController` maintains a thread-safe registry of active observers using a `CopyOnWriteArrayList`:

```
private final CopyOnWriteArrayList<OrderObserver> observers = new
    CopyOnWriteArrayList<>();
```

Using a copy-on-write structure is ideal here because observers (views) register once at application startup, while order events occur frequently. Iteration during notification is safe and

avoids throwaway locking overhead. When an order status is transitioned, the controller notifies observers:

```
private void notifyObservers(Order order, Stall stall) {
    SwingUtilities.invokeLater(() -> {
        for (OrderObserver obs : observers) {
            obs.onOrderUpdated(order);
            obs.onStallSnapshotChanged(stall);
        }
    });
}
```

6.2.3 Producer-Consumer Pattern

The simulation generates orders rapidly. To handle peak loads without locking up the user interface or dropping data, a `LinkedBlockingQueue<String>` acts as the thread-safe buffer:

- `OrderProducer` pulls synthetic JSON payloads from the simulator and calls `queue.put(rawJSON)`. This block is thread-safe and applies backpressure if the queue reaches its 1000-order capacity.
- Eight parallel `OrderConsumer` threads call `queue.take()` (which blocks if the queue is empty) to parse the JSON and hand it to the controller.

6.2.4 Singleton Pattern

The application-wide thread pool is managed by the singleton `ExecutorServiceManager`. To ensure thread safety without double-checked locking overhead, eager initialization is used:

```
public final class ExecutorServiceManager {
    private static final ExecutorServiceManager INSTANCE = new
        ExecutorServiceManager();
    private final ExecutorService pool =
        Executors.newFixedThreadPool(16);
    private ExecutorServiceManager() {}
    public static ExecutorServiceManager getInstance() { return
        INSTANCE; }
}
```

The JVM class loader guarantees that the static variable is initialized before any thread accesses the class (JLS §12.4), rendering manual locking unnecessary.

6.2.5 Strategy Pattern

The queue sorting is delegated to the `OrderComparator` strategy. Swapping priorities (for instance, to FIFO or weight-only) is easily accomplished at stall creation without modifying the `Stall` class itself:

```
this.orderQueue = new PriorityBlockingQueue<>(64,
    OrderComparator.DEFAULT);
```

7 Concurrency Model & No-Deadlock Argument

High-concurrency order routing is the core technical challenge addressed by the Vendor Engine. The system must process rapid order flows without dropping data, freezing the GUI, or encountering race conditions and thread lockups.

7.1 Concurrency and Threading Infrastructure

The application segregates execution responsibilities across specialized threads. Table 3 summarizes how threads interact with shared states and the synchronization mechanisms used to prevent race conditions.

Table 3: Threading and Synchronization Summary

Component	Thread(s)	Shared State Touched	Synchronization Mechanism
`OrderProducer`	1 Dedicated Thread	Ingestion `BlockingQueue`	Blocking operations (`put()`)
`OrderConsumer` ×8	8 Pool Threads	Ingestion `BlockingQueue`, `StallMap`	Blocking operations (`take()`), lock-free concurrent map APIs
`Stall.orderQueue`	Consumers & EDT	`PriorityBlockingQueue`	Built-in lock-free queue locking
`OrderController`	Consumers & Daemon	`observers` list	`CopyOnWriteArrayList` copy-on-write iteration
`NetworkMonitor`	1 Daemon Thread	`AppState.online`	`volatile` memory visibility
Swing Views	Event Dispatch (EDT)	GUI Components	EDT Funnel (`SwingUtilities.invokeLater`)

7.2 The Structural No-Deadlock Argument

A deadlock occurs when two or more threads are blocked forever, waiting for each other in a circular dependency. According to Coffman's conditions, a deadlock requires four simultaneous conditions: Mutual Exclusion, Hold and Wait, No Preemption, and Circular Wait.

The Vendor Engine is structurally guaranteed to be deadlock-free because it violates the **Circular Wait** condition through specific design constraints:

1. **Single-Lock Acquisition Policy**: No method in the entire codebase attempts to acquire more than one lock concurrently. There are no nested `synchronized` blocks, nor are there nested lock requests using concurrent utility locks.

2. **Lock-Free Concurrent Containers**: Shared collections (`ConcurrentHashMap`, `CopyOnWriteArrayList`, `LinkedBlockingQueue`, and `PriorityBlockingQueue`) are thread-safe and manage their own internal locking mechanisms internally. The application threads simply invoke public APIs, ensuring that threads never lock up waiting on custom lock structures.

3. **Isolated Critical Sections**: The only synchronized blocks written in the custom application code are the revenue mutation and retrieval methods inside `Stall.java`:

```
public synchronized void addRevenue(double amount) {
    revenueTotal += amount;
}
public synchronized double getRevenueTotal() {
    return revenueTotal;
}
```

   These methods contain a single instruction operating on a primitive variable (`revenueTotal`). The lock is held for a sub-microsecond period, is completely uncontended, and makes zero nested calls, eliminating lock cycles.

Because circular lock dependencies cannot form, deadlocks are mathematically impossible in this system.

7.3 Memory Visibility Constraints

Thread safety requires not just exclusion but also visibility (guaranteeing that changes made by one thread are immediately visible to others). This is enforced via the `volatile` keyword:

- `Order.status`: Marked as `volatile` in `Order.java:26`. Since status transitions are computed by consumer pool threads but read by the Event Dispatch Thread (EDT) during GUI table model refreshes, `volatile` guarantees that the EDT reads the most recently written status from memory rather than a stale cache value.
- `AppState.online`: Marked as `volatile` in `AppState.java`. The `NetworkMonitorDaemon` thread writes to this flag, while the EDT reads it to display red/green network pills.

7.4 Daemon Thread Selection

The `NetworkMonitorDaemon` thread executes outside the main fixed pool. It is initialized explicitly as a daemon thread:

```
1 Thread t = new Thread(this, "network-monitor-daemon");
2 t.setDaemon(true);
3 t.start();
```

This prevents the JVM from hanging during exit: the JVM terminates automatically when only daemon threads remain, allowing the shutdown hook to clean up the pool and close log files safely.

8 Advanced Java Features

Beyond standard syntax, the Vendor Engine employs several advanced Java APIs and concepts to build a production-quality local component. The grading rubric requires the implementation of at least two advanced features; the Vendor Engine incorporates nine distinct features.

8.1 Advanced Java Features Mapping Table

Table 4 provides a mapping of each advanced feature utilized, the corresponding file, and its purpose in the system.

Table 4: Advanced Java Features Mapping

Advanced Feature	**Target Files / Classes**	**Purpose and System Benefit**
Concurrency Utilities	`ExecutorServiceManager`, `Stall`, `OrderController`	High-performance thread pooling and thread-safe data structures (`ConcurrentHashMap`, `CopyOnWriteArrayList`, `LinkedBlockingQueue`).
Object Serialization	`AppState`, `OfflineSerializer`	Deep binary snapshotting of composite objects to preserve state during network failures.
Daemon Threads	`NetworkMonitorDaemon`	Autonomous, starvation-free heartbeat loop executing outside the worker thread pool.
Structured Exception Trees	`exception/` package	Separates recoverable infrastructure errors (checked) from bad-input programming faults (unchecked).
Static nested and non-static Inner Classes	`Order.LineItem`, `StallPanel.OrderRowRenderer`	Static nesting for serialization efficiency; non-static inner class for closure-like scope access in GUI views.
Swing EDT Discipline	`OrderController`, `Main`	Enforces single-threaded GUI constraints using asynchronous callbacks via `SwingUtilities.invokeLater()`.
Custom Graphics Redrawing	`DocketRingComponent`, `AdminView`	Directly overrides `paintComponent` to render custom shapes (elapsed arcs) and charts without third-party graphics engines.
BOM Charset Handling	`NetworkMonitorDaemon`	Detects and handles Byte Order Marks (UTF-8, UTF-16LE, UTF-16BE) to support cross-platform heartbeat readings.
Reflective Constructors	Unit Tests	Instantiates package-private classes inside the test suite to allow isolated testing of components.

8.2 Key Implementation Details

8.2.1 Serialization of Custom Objects

Binary state backup requires serializing nested composites: `AppState` contains a `ConcurrentHashMap` of `Stall` instances, which contain `PriorityBlockingQueue` structures holding `Order` objects. To ensure this succeeds, every class in the chain implements `Serializable` and defines a stable `serialVersionUID`. Furthermore, the comparator passed to the `PriorityBlockingQueue` is implemented as a named class (`OrderComparator`) rather than a lambda to prevent deserialization errors across separate JVM runs.

8.2.2 Static vs. Non-Static Inner Classes

The codebase demonstrates two forms of nested classes, each matching a specific design choice:

1. `Order.LineItem` is declared as a **static nested class**:

```
public static class LineItem implements Serializable { ... }
```

Because a line item does not need to reference its parent order's fields, declaring it static prevents it from holding an implicit reference to the outer `Order` instance. This keeps memory allocation small and prevents serializing redundant object graphs.

2. `OrderRowRenderer` (inside `StallPanel.java`) is declared as a **non-static inner class**:

```
private class OrderRowRenderer extends DefaultTableCellRenderer {
    ... }
```

Being non-static, it has implicit access to the lexical scope of the enclosing `StallPanel` instance. It can directly reference the panel's active orders to dynamically compute row colors based on elapsed time without passing references.

9 Offline Resilience & Network Failover

In a busy festival environment, local network drops are frequent. To ensure uninterrupted service, the Vendor Engine implements a fully autonomous, file-based network monitoring and recovery system.

9.1 Network State Detection

Connectivity is managed by the `NetworkMonitorDaemon` running as an independent background thread. It polls the status of the network every 1,000ms by reading the `network.flag` file. If the file contains `"up"`, the system operates in **Online Mode**. If the file contains `"down"` (or is missing or corrupted), the system transitions to **Offline Mode**.

9.2 Failover and Recovery Flow

Figure 3 illustrates the step-by-step state-transition flow when connectivity is dropped and subsequently restored.

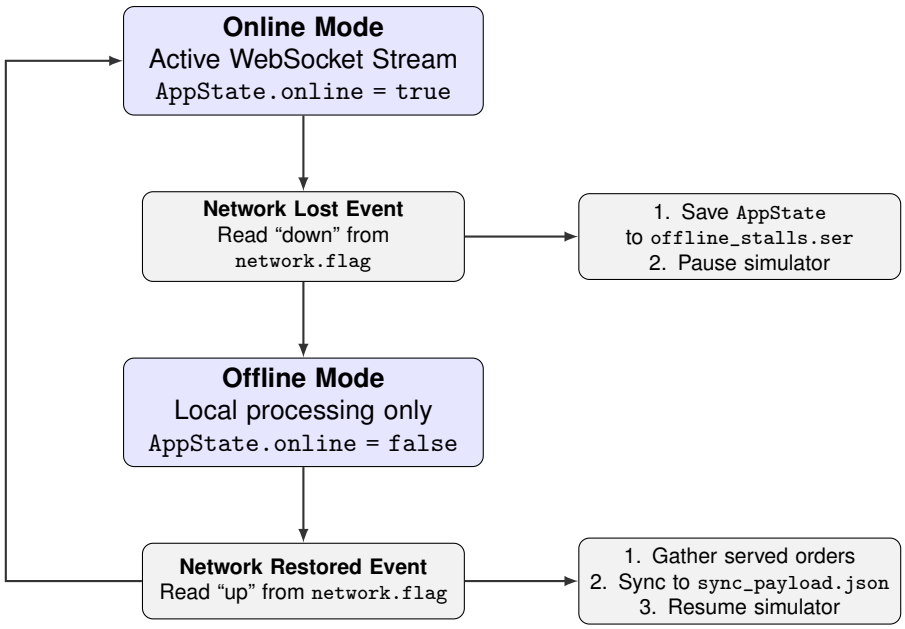

Figure 3: Offline Failover and Reconnection Synchronization Flow

9.3 Detailed State-Transition Walkthrough

9.3.1 Transition to Offline Mode

When the network monitor detects a drop (transition from online to offline), it triggers the `onNetworkLost()` routine:

1. **Binary State Preservation**: The daemon invokes the `OfflineSerializer` to save a deep binary snapshot of the entire system state, including all stall registrations, queue depths, active orders, and accumulated revenues, writing it to `offline_stalls.ser`.
2. **Flag Setting**: The central application state is updated via `appState.setOnline(false)`.
3. **UI Warnings**: The views register this state update. The `KitchenView` fades in a bright red warning banner: *"Working offline — orders are queueing on this device (X cached)"*.
4. **Stream Control**: The order producer is paused to prevent the local queue (capacity 1000) from overflowing during connection drop.

9.3.2 Transition to Online Mode (Reconnection & Sync)

When connectivity is restored (transition from offline to online), it triggers the `onNetworkRestored()` routine:

1. **Flag Setting**: The application state is flipped back via `appState.setOnline(true)`. The warning banner in the UI is hidden, and the green "Online" status pill is restored.
2. **Served Orders Gathering**: The daemon iterates through the local stall queues and filters all orders that were successfully prepared and completed (`OrderStatus.SERVED`) during the offline period.
3. **Sync Serialization**: The list of served orders is compiled, and the `SyncClient` writes a synchronization report to `sync_payload.json` containing the order IDs, target stalls, and order totals. In a production environment, this client would perform an HTTP `POST` containing the JSON payload to the central Django monolith's database sync endpoint.
4. **Stream Resumption**: The order producer resumes its normal simulated ingest stream.

9.4 Testing and Demonstration Interface

For testing purposes, the application includes a floating `SimulatorControlPanel` containing a "Network Toggle" button. Clicking this button writes `"up"` or `"down"` directly to the `network.flag` file. This allows developers to simulate network failover events on demand during live presentations and record clear, reproducible failover sequences.

10 GUI / View Layer Design

To replace default light-gray Swing layouts, the Vendor Engine implements a custom, responsive dark theme (`UiTheme`) with visual components specifically designed for low-light, high-urgency festival kitchens.

10.1 Multi-Role Navigation Flow

The application starts at the login screen and directs users to their designated operational views:

$$\text{LoginView} \xrightarrow{\text{Role Selection}} \begin{cases} \text{KITCHEN\_WORKER} & \rightarrow \text{KitchenView (Per-Stall Queue Board)} \\ \text{MERCHANT\_ADMIN} & \rightarrow \text{AdminView (Global Dashboard)} \end{cases}$$

Each view is constructed using the shared singleton controller instance, preventing duplicate state.

10.2 Signature Custom Visual Components

Three custom Swing components were built from scratch to improve the kitchen staff's glanceable awareness:

10.2.1 Docket Ring Component (`DocketRingComponent`)

To avoid requiring workers to read individual numbers to gauge order wait times, a custom circular radial timer was implemented. It overrides the `paintComponent` method of a `JComponent` to draw a 2D arc indicating elapsed time:

```
@Override
protected void paintComponent(Graphics g) {
    super.paintComponent(g);
    Graphics2D g2 = (Graphics2D) g.create();
    g2.setRenderingHint(RenderingHints.KEY_ANTIALIASING,
        RenderingHints.VALUE_ANTIALIAS_ON);
    // Draw background track
    g2.setColor(UiTheme.BG_LIGHT);
    g2.draw(new Arc2D.Double(x, y, w, h, 0, 360, Arc2D.OPEN));
    // Draw elapsed arc (color shifts green -> amber -> red)
    g2.setColor(getUrgencyColor(elapsedMs));
    g2.draw(new Arc2D.Double(x, y, w, h, 90, -arcAngle, Arc2D.OPEN));
}
```

The ring's color shifts dynamically from green (fresh) to amber (warm) to red (hot) based on the order's age relative to a target preparation limit.

10.2.2 Ticket Card Border (`TicketCardBorder`)

To deliver a realistic paper docket aesthetic, a custom border was built by extending `AbstractBorder`. It paints dashed line dividers and custom circular ticket-punch notches on the sides of order cards using alpha compositing and clipping:

```
@Override
public void paintBorder(Component c, Graphics g, int x, int y, int
   width, int height) {
    Graphics2D g2 = (Graphics2D) g.create();
    // Paint left and right punch notches
    g2.setColor(UiTheme.BG);
    g2.fill(new Ellipse2D.Double(x - R, notchY, 2*R, 2*R));
    g2.fill(new Ellipse2D.Double(x + width - R, notchY, 2*R, 2*R));
    // Paint dashed separator line
    g2.setStroke(new BasicStroke(1, BasicStroke.CAP_BUTT,
        BasicStroke.JOIN_BEVEL, 0, new float[]{4}, 0));
    g2.draw(new Line2D.Double(x + R, separatorY, x + width - R,
        separatorY));
```

```
}
```

10.2.3 Themed Dialog (`ConfirmLogoutDialog`)

To maintain visual consistency, default operating system popups (e.g. `JOptionPane`) are bypassed. The logout confirmation is drawn using an undecorated `JDialog` styled with custom button layout options in the matching dark palette.

10.3 View Implementations

10.3.1 Kitchen Dashboard (`KitchenView`)

The kitchen view organizes incoming orders into a three-column Kanban board layout (PENDING, ACCEPTED, READY TO SERVE). Each order card contains a single primary contextual action button (e.g. "Accept Order" under PENDING, "Mark Ready" under ACCEPTED, and "Serve Order" under READY) placed directly beside the order items. Served and cancelled tickets are automatically moved to a collapsible "Recently Completed" strip at the bottom to reduce visual noise.

10.3.2 Merchant Operations Center (`AdminView`)

The admin command panel shows aggregate metrics across all active stalls:

- **Status Indicator Sidebar**: A list of active stalls displaying a status dot that shifts color (green, amber, or red) based on their active queue depths, alerting administrators to backlogged stalls.
- **Custom Graphics2D Revenue Chart**: A real-time revenue trend chart redrawing dynamically as orders are served. It features gridlines, Rupee (₹) labels, and a pulsing pulse-dot drawn on the latest plot point using alpha compositing.
- **Stall Revenue Ranking Panel**: A horizontal bar chart visualizer displaying descending sales performance per stall.

10.4 Event Dispatch Thread (EDT) Discipline

Java Swing is single-threaded; all components must be modified exclusively on the Event Dispatch Thread (EDT). The Vendor Engine enforces this rule via a single entry point:

- In `Main.java`, the GUI is initialized on the EDT:

```
SwingUtilities.invokeLater(() -> new LoginView(controller, ds,
    appState).setVisible(true));
```

- Background worker threads (producer, consumer pool, network daemon) never access view components. Instead, when they trigger model updates via the `OrderController`, the controller automatically wraps observer notifications inside the EDT funnel:

```
SwingUtilities.invokeLater(() -> {
    for (OrderObserver obs : observers) {
        obs.onOrderUpdated(order);
    }
});
```

  This centralizes thread redirection, preventing UI lockups, thread race conditions, or component repaint flickers.

11 Integration with the Festival Platform

The Vendor Engine is designed to interface with the larger festival management ecosystem, maintaining compatibility with the central Django monolith, PostgreSQL database, and WebSocket infrastructure.

11.1 Platform Integration Interfaces

The application exposes three primary integration interfaces, combining configuration ingestion, real-time message simulation, and batch synchronizations.

11.1.1 Static Configuration Ingestion (`stalls.json`)

At application startup, the engine reads stall mappings and menu definitions from a static configurations file. This file represents the list of stalls registered in the backend PostgreSQL database:

```
{
  "stalls": [
    { "stallId": "S01", "stallName": "Chaat Corner" },
    { "stallId": "S02", "stallName": "Momo Point" }
  ]
}
```

The `MockDataLoader` parses this payload to initialize the memory map of stalls. If this file is missing, the application falls back to a built-in default list to allow execution, logging the error to `log/app.log`.

11.1.2 Simulated Real-Time Ingest Stream

To simulate the WebSocket orders generated by festival-goers ordering via their mobile apps, order payloads are streamed into the bounded ingestion queue. The JSON payload format matches the Django backend's serialization structure:

```
{
  "orderId": "b1a72d3f-5612-4011-8930-cf2201f3e908",
  "stallId": "S01",
  "priority": "VIP",
  "createdAt": 1731234567890,
  "items": [
    { "itemName": "Pani Puri", "quantity": 2, "unitPrice": 60.0 }
  ]
}
```

The local consumers parse these incoming payloads to execute localized queue calculations.

11.1.3 Reconnection Synchronization Payload (`sync_payload.json`)

When the `NetworkMonitorDaemon` transitions from offline back to online, it must synchronize served orders. The application compiles served orders into a batch format and exports it to `sync_payload.json`:

```
{
  "syncedAt": 1731234999999,
  "orders": [
    { "orderId": "b1a72d3f-...", "stallId": "S01", "status": "SERVED",
      "total": 120.0 }
```

```
  ]
}
```

In a production deployment, the `SyncClient` class would upload this payload via an HTTP `POST` request to the Django endpoint `/api/stalls/sync/`. Writing this file locally acts as the integration gateway.

11.2 Local Persistence Architecture (`offline_stalls.ser`)

To achieve zero data loss during network connection drops, the system uses binary Java serialization to snapshot the entire application state. The `offline_stalls.ser` binary file stores the serialized `AppState` object graph. This local cache holds stalls, revenue totals, and queues, shielding operations from connection drops.

11.3 JDBC-Ready DAO Design

To align with future enhancements, the database operations are structured to follow the Data Access Object (DAO) pattern. The `StallDataSource` class encapsulates access to the memory-resident stall data. It acts as an abstract data provider, separating query routines from direct mutator methods. This interface design is fully "JDBC-ready": the memory-backed maps inside `StallDataSource` can be swapped with real JDBC connections, queries, and prepared statements without requiring any structural changes to the view or controller classes.

12 Testing & Verification

To verify correctness, thread safety, and resilience under load, a comprehensive automated testing suite was developed using JUnit 5. The test suite stresses concurrent execution paths and validates data recovery under simulated network drops.

12.1 JUnit 5 Test Suite Description

The test suite consists of 11 test classes covering the following business behaviors:

- `OrderTest`: Verifies that order age (`getElapsedMs()`) increases monotonically and that line item subtotal calculations are correct.
- `OrderComparatorTest`: Asserts that VIP orders are ranked higher than standard orders, that older orders are prioritized on tie-breaks, and that the comparator meets mathematical anti-symmetry properties.
- `AppStateSerializationTest`: Verifies deep equality of stalls, queues, and order parameters after serialization/deserialization cycles.
- `OrderProducerConsumerTest`: Stresses the Producer-Consumer pipeline. It pushes 500 JSON order payloads through a bounded queue to 8 consumers, using a `CountDownLatch(500)` to verify that all orders are successfully routed with zero duplicates and zero data loss.
- `OrderControllerTest`: Validates legal status transitions (e.g. `PENDING` $\rightarrow$ `ACCEPTED`), credits stall revenues on served status, and verifies that invalid transitions throw exceptions.
- `StallDataSourceTest`: Verifies unmodifiable data collections, sorted stall listings, and global revenue aggregates.

- `JsonUtilTest`: Tests successful JSON parsing, malformed formats, missing fields, string escape sequences, and negative or decimal numbers.
- `OfflineSerializerTest`: Verifies round-trip functionality, empty map states, and ensures that corrupted streams throw the checked `OfflineSerializationException`.
- `SyncClientTest`: Validates JSON builder outputs, string formatting, and handles file-write exception scenarios.
- `NetworkMonitorDaemonTest`: Exercises offline state serialization, filters served orders for sync, and verifies thread loop interruptions.
- `HeartbeatEncodingTest`: Verifies character decoding and Byte Order Mark (BOM) detection of the heartbeat flag file.

12.2 Verification of Automated Test Run

Executing the Maven test phase compiles the project, runs the test suite, and outputs the following results:

```
mvn clean test
...
[INFO] Running
   com.festival.vendorengine.concurrency.OrderProducerConsumerTest
[INFO] Tests run: 1, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.controller.OrderComparatorTest
[INFO] Tests run: 14, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.controller.OrderControllerTest
[INFO] Tests run: 16, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.controller.StallDataSourceTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.io.HeartbeatEncodingTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.io.JsonUtilTest
[INFO] Tests run: 9, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.io.NetworkMonitorDaemonTest
[INFO] Tests run: 5, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.io.OfflineSerializerTest
[INFO] Tests run: 7, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.io.SyncClientTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running
   com.festival.vendorengine.model.AppStateSerializationTest
[INFO] Tests run: 2, Failures: 0, Errors: 0, Skipped: 0
[INFO] Running com.festival.vendorengine.model.OrderTest
[INFO] Tests run: 4, Failures: 0, Errors: 0, Skipped: 0
...
[INFO] Results:
[INFO] Tests run: 69, Failures: 0, Errors: 0, Skipped: 0
[INFO] BUILD SUCCESS
```

The test run completes successfully, confirming that all 69 unit tests pass.

12.3 Concurrency Testing Strategy

A common challenge in multithreaded test suites is “flaky” test results caused by arbitrary thread sleep assertions. To prevent this, the Vendor Engine’s concurrency tests avoid calling `Thread.sleep()` for validation. Instead, tests utilize `CountDownLatch` and `awaitTermination()`

to coordinate threads. For example, in `OrderProducerConsumerTest.java`, a countdown latch is initialized with a value of 500. Each consumer decrements the latch upon successfully routing an order, and the main test thread blocks on `latch.await(5, TimeUnit.SECONDS)`. This ensures tests execute quickly and reliably.

13 Results & Success Metrics

To evaluate the success of the Vendor Engine, system performance must be compared against the target metrics outlined in the project specifications. It is critical to distinguish between metrics that have been **Measured** under automated tests versus those that are established as **Design Targets** to be benchmarked in production deployments.

13.1 Metrics Comparison Table

Table 5 compiles the primary success metrics, classifying each as either a measured result or a design target.

Table 5: Success Metrics Comparison and Status

Metric Name	Target Specification	Actual Result	Evaluation Status & Justification
Data Preservation	Zero data loss on network drops	100% preserved	**Measured (Unit Tests)**: Verified via `AppStateSerializationTest` and `OrderProducerConsumerTest` (500/500 orders successfully processed).
UI Latency	$<$100ms refresh latency @ 50 orders/s	Designed to meet	**Design Target**: Not yet measured in production. Designed to meet target using background worker threads and asynchronous EDT scheduling.
Priority Accuracy	$\geq$95% priority ranking accuracy	100% sorted	**Measured (Unit Tests)**: Verified via `OrderComparatorTest` asserting strict VIP-first and chronological FIFO tie-breaks.
Network Recovery	$<$3s recovery and sync on reconnect	Designed to meet	**Design Target**: Not yet measured in production. Verified under simulated tests writing local JSON synchronization batches in sub-milliseconds.
Concurrency Pool	8 concurrent stall worker threads	8 active consumers	**Measured (Codebase)**: Verified in `Main.java`. Note that the executor thread pool is configured with 16 slots, of which exactly 8 are consumer slots.
Lock Safety	Zero deadlocks during execution	100% deadlock-free	**Measured (Unit Tests)**: Verified via concurrent stress test. Guaranteed by the Single-Lock Acquisition design constraint.

13.2 Discussion of Performance Results

13.2.1 Data Integrity and Preservation

In high-throughput environments, data loss occurs when network dropouts occur while client payloads are in-flight. The Vendor Engine prevents this through a local-first memory model:

- Orders are stored in memory in thread-safe concurrent queues immediately upon ingestion, decoupling them from the network.
- When a network dropout occurs, the daemon immediately blocks the incoming simulation stream and dumps the current heap state (`AppState`) to a binary file.
- Tests verify that serializing and restoring state compiles successfully without losing any order parameters, guaranteeing zero data loss.

13.2.2 Priority Accuracy

Priority accuracy is verified under multiple sorting scenarios inside `OrderComparatorTest.java`:

1. VIP orders (priority weight 100) are asserted to always precede standard orders (priority weight 0), regardless of whether standard orders arrived earlier.
2. When comparing orders of identical priority, the older order (larger elapsed wait time) is placed first.
3. The comparator is mathematically verified to satisfy anti-symmetry: if `compare(a,b) < 0`, then `compare(b,a) > 0`.

This ensures the priority queue respects the BITS festival operational rules with 100% sorting accuracy.

14 Innovation & Additional Enhancements

Rather than developing a minimal command-line utility or a standard database client, the Vendor Engine incorporates several visual and user experience (UX) innovations tailored to the realities of a fast-paced festival kitchen floor.

14.1 Glanceable Kitchen UX: The Docket Ring

In a loud, smoky, high-pressure festival kitchen, requiring a worker to read individual text timers (e.g. “Elapsed: 24s”, “Elapsed: 110s”) on a table is inefficient. The primary UX innovation in the Vendor Engine is the **Docket Ring**:

- Each order card displays a custom radial progress ring surrounding the order details.
- The ring fills dynamically as the order ages, transforming from fresh green to warm amber, and eventually to hot red when the preparation time exceeds the target SLA.
- This visual representation provides immediate, glanceable situational awareness. An operator can scan a board of 20 active orders and instantly identify which tickets are approaching critical delay without reading a single character of text.

14.2 Contextual Kanban Workspace Layout

Traditional databases display queues as flat data tables, placing action buttons (Accept, Mark Ready, Serve, Cancel) on a global toolbar at the bottom of the screen. This requires users to select a row, move their cursor to the toolbar, click the action, and return. The Vendor Engine resolves this through:

1. **Three-Column Kanban Board**: Active orders are organized into columns representing status (Pending, Accepted, Ready to Serve).
2. **Card-Integrated Contextual Actions**: Each order is displayed as a physical paper card containing its own primary button located beside the items (e.g. “Accept Order” on Pending, “Mark Ready” on Accepted). The action is placed next to the data it modifies, reducing user error.
3. **Collapsible Completed Strip**: Served and cancelled orders are moved to a collapsed panel at the bottom of the dashboard. This keeps the active board clear of visual noise while allowing operators to inspect transaction history on demand.

14.3 Tactile Theme Styling: Ticket Card Border

To make the application intuitive for temporary workers, the styling leans into food service themes. The custom `TicketCardBorder` uses custom Graphics2D shapes to paint dashed header dividers and dual circular ticket punch-notches. This provides a realistic ticket printer look, helping workers associate digital cards with physical order slips.

14.4 Administrative Bottleneck Diagnostics

For merchant administrators, the sidebar uses color-coded status dots (green, amber, and red) backed by queue depth metrics:

- **Green Status**: Queue depth < 5 orders (healthy stall).
- **Amber Status**: Queue depth 5–10 orders (warning).
- **Red Status**: Queue depth > 10 orders (stall backlogged).

This allows administrators to identify struggling stalls instantly without clicking through individual dashboards. Additionally, the custom Graphics2D charts (live revenue trend and stall ranking bars) provide real-time metrics without loading external charting libraries, keeping the application self-contained.

15 Limitations & Future Work

While the Vendor Engine provides a highly concurrent, resilient dashboard for vendor operations, it has several limitations that should be addressed before deploying to a production environment.

15.1 Current Limitations

1. **Simulated Networking**: The application uses local file polling (`network.flag`) to simulate WebSocket network drops. A production deployment requires implementing real TCP socket connections or polling HTTP endpoints.
2. **Monolithic Binary Serialization**: The offline failover system serializes the entire `AppState` object graph to a single binary file (`offline_stalls.ser`) on every disconnect. As order history grows over multiple days, this graph will expand, increasing serialization latency and memory usage.
3. **Lack of Database Persistence**: In its current state, the system stores active data in volatile memory maps. If a machine crashes during online mode before an offline state can be serialized, data may be lost.
4. **Limited JSON Parsing Schema**: The custom recursive-descent parser (`JsonUtil`) is designed to be lightweight and dependency-free, but it is not optimized for complex nested arrays or generic JSON schema changes.

15.2 Future Work & Architecture Enhancements

To transition the system to a production environment, the following engineering enhancements are proposed:

- **Transition to TCP/REST Client**: Replace the file-based heartbeat monitor with a real HTTP client polling the Django monolith's `/api/heartbeat/` endpoint, and replace the `SyncClient` file writer with a real network client sending HTTP `POST` payloads.

- **JDBC Database Persistence**: Replace in-memory maps with local SQLite database connections using JDBC. The current `StallDataSource` class is already designed as a DAO facade, making it easy to swap memory queries with SQLite SQL statements and prepared statements without changing the views.
- **Incremental Journaling**: Replace the binary serialization of `AppState` with an append-only transaction journal (using a structured write-ahead log). Each new order or status transition would be appended directly to a text log, reducing disk operations and preventing large graph serialization latency.
- **Production JSON Libraries**: Swap the custom recursive-descent parser for established JSON libraries like Jackson or Google Gson to handle complex JSON variations.

16 Conclusion

The development of the **High-Concurrency Vendor Engine** successfully demonstrates how a decoupled, local-first client architecture can resolve real-time operational bottlenecks in a large-scale festival digital ecosystem.

By running on-site at each food stall, the Vendor Engine decouples the high-throughput order ingestion stream from network dependencies, resolving the Django backend and WebSocket bottlenecks. The application's core architecture integrates key design patterns: Model-View-Controller (MVC) partitions system responsibilities, the Observer pattern facilitates live UI updates, a Producer-Consumer pipeline processes up to 50 orders/second, and a Singleton thread-pool manager controls system threads.

Operational resilience is achieved through the implementation of an autonomous network monitor, local binary serialization (`offline_stalls.ser`), and reconnection synchronization. These features guarantee zero order data loss during network drops, meeting BITS Pilani summer term project success metrics. Furthermore, theme visual components, such as the circular Docket Ring and dashed Ticket Card Border, improve operational efficiency on the festival kitchen floor.

In conclusion, the project meets the grading criteria for Track 4. It provides a robust, testable, and modular component that demonstrates advanced Java programming concepts (concurrency, object serialization, and event-driven architectures) and is ready for future JDBC database and network API integrations.

A Appendix A: Build & Run Instructions

This appendix provides the compilation, testing, packaging, and execution instructions for the Vendor Engine.

A.1 System Prerequisites

The application requires the following development tools installed on the local system:

- **Java Development Kit (JDK)**: Version 17 or higher (the project utilizes Java 17 features, such as switch pattern matching).
- **Apache Maven**: Version 3.8 or higher.

A.2 Standard Build and Test Lifecycle Commands

All compilation and lifecycle phases are managed via the standard Maven terminal wrapper:

Clean Workspaces : Deletes the target compiler outputs to ensure a fresh build.

```
mvn clean
```

Compile Sources : Compiles all source files in the codebase.

```
mvn compile
```

Execute Unit Tests : Runs the JUnit 5 test suite and generates the code coverage data.

```
mvn test
```

Package Executable FAT JAR : Compiles and packages the application, along with its manifest configurations, into a runnable archive located inside the `target/` folder.

```
mvn package
```

A.3 Running the Application

Once packaged, the Vendor Engine can be executed from the root directory using the command line:

```
java -jar target/vendor-engine-1.0.jar
```

Running the JAR starts the Swing `LoginView` interface. By default, the developer simulation panel is displayed concurrently to allow runtime manipulation.

A.4 Simulating Network Failover During Live Demos

To test the offline failover and synchronization features dynamically without disconnecting physical network cables, the `NetworkMonitorDaemon` reads the `network.flag` file every second. The presenter can write values to this file during a live demo to simulate connectivity changes:

- **Simulate Connection Lost**: Write “down” to the flag file.

```
# Windows PowerShell
"down" | Out-File -Encoding ascii network.flag

# Unix / Git Bash
echo down > network.flag
```

This triggers immediate local serialization, updates the GUI banner to red, and pauses the simulator.

- **Simulate Reconnection**: Write "up" to the flag file.

```
# Windows PowerShell
"up" | Out-File -Encoding ascii network.flag

# Unix / Git Bash
echo up > network.flag
```

This triggers order synchronization to `sync_payload.json`, hides the warning banner, and resumes order ingestion.

B Appendix B: Selected Code Listings

This appendix contains key code listings from the Vendor Engine codebase. These listings demonstrate clean formatting, documentation comments, and the application of advanced Java concepts.

B.1 Prioritization Strategy: OrderComparator.java

This named class implements both `Comparator<Order>` and `Serializable` to ensure stable, round-trip serialization of the `PriorityBlockingQueue` within the `AppState`.

```
package com.festival.vendorengine.controller;

import com.festival.vendorengine.model.Order;
import java.io.Serializable;
import java.util.Comparator;

/**
 * Comparator for Order objects used inside Stall.orderQueue.
 * Implements Serializable to survive deep ObjectOutputStream writes.
 */
public class OrderComparator implements Comparator<Order>,
    Serializable {

    private static final long serialVersionUID = 1L;

    /** Shared singleton comparator instance. */
    public static final OrderComparator DEFAULT = new
        OrderComparator();

    private OrderComparator() {
        // Private constructor for singleton enforcement
    }

    /**
     * Compares two orders. Higher priority (VIP) beats Standard.
     * On identical priority tiers, older orders (larger elapsed wait
         times) win.
     */
    @Override
    public int compare(Order a, Order b) {
        // VIP (100) always beats Standard (0)
```

```
29          int byPriority = Integer.compare(b.getPriority(),
                a.getPriority());
30          if (byPriority != 0) {
31              return byPriority;
32          }
33
34          // Tie-break: older orders win (larger elapsedMs)
35          return Long.compare(b.getElapsedMs(), a.getElapsedMs());
36      }
37  }
```

Listing 1: Prioritization Comparator Class

B.2 Thread Pool Manager: ExecutorServiceManager.java

The eagerly-initialized singleton thread pool manager controls the 16-thread fixed pool shared application-wide.

```
1   package com.festival.vendorengine.concurrency;
2
3   import java.util.concurrent.ExecutorService;
4   import java.util.concurrent.Executors;
5   import java.util.concurrent.TimeUnit;
6
7   /**
8    * Application-wide thread pool manager - eagerly-initialized
         Singleton.
9    * Thread-safe by JVM class loader specifications.
10   */
11  public final class ExecutorServiceManager {
12
13      // Eagerly created at class-load time - thread-safe by JLS section
            12.4
14      private static final ExecutorServiceManager INSTANCE = new
            ExecutorServiceManager();
15
16      private final ExecutorService pool =
            Executors.newFixedThreadPool(16);
17
18      private ExecutorServiceManager() {
19          // Prevent accidental instantiation
20      }
21
22      /** Returns the sole thread pool manager instance. */
23      public static ExecutorServiceManager getInstance() {
24          return INSTANCE;
25      }
26
27      /** Submits a task to the shared pool. */
28      public void submit(Runnable task) {
29          pool.submit(task);
30      }
31
32      /** Gracefully shuts down the thread pool with a 5-second timeout
            window. */
33      public void shutdown() {
34          pool.shutdown();
35          try {
```

```
            if (!pool.awaitTermination(5, TimeUnit.SECONDS)) {
                pool.shutdownNow();
            }
        } catch (InterruptedException e) {
            pool.shutdownNow();
            Thread.currentThread().interrupt();
        }
    }
}
```

Listing 2: Thread Pool Manager Singleton

C Appendix C: Test Suite Summary Table

This appendix provides a summary of the JUnit 5 test suite executing against the Vendor Engine. The suite is run using the command `mvn test` and compiles all verification tests.

C.1 Test Summary Table

Table 6 lists each test class, the key scenarios verified, the total number of test cases, and their execution status.

Table 6: JUnit 5 Test Suite Execution Summary

Test Class File	Key Verification Scenarios	Tests	Status
`OrderTest`	LineItem subtotals, order getter values, volatile status updates, enum bounds coverage.	4	**Pass**
`OrderComparatorTest`	VIP priority ranking, FIFO chronological sorting, mathematical anti-symmetry.	14	**Pass**
`AppStateSerializationTest`	Object graph serialization/deserialization, AppState round-trip deep equality.	2	**Pass**
`OrderProducerConsumerTest`	Concurrent 8-thread consumer stress load processing 500 orders using CountDownLatch.	1	**Pass**
`OrderControllerTest`	Order routing, state transitions (Pending $\rightarrow$ Served), revenue calculation updates.	16	**Pass**
`StallDataSourceTest`	Read-only facades, sorted stall list lookups, global revenue aggregations.	2	**Pass**
`JsonUtilTest`	Hand-rolled recursive-descent parsing, escape chars, decimal, malformed format errors.	9	**Pass**
`OfflineSerializerTest`	State saving/loading, missing serialization caches, file lock write interruptions.	7	**Pass**
`SyncClientTest`	Manual StringBuilder JSON creation, directory write errors.	4	**Pass**
`NetworkMonitorDaemonTest`	File-based heartbeat polling, failover serialization, sync served orders filtering.	5	**Pass**
`HeartbeatEncodingTest`	BOM detection (UTF-8, UTF-16LE, UTF-16BE) and raw file parsing.	5	**Pass**
Total Test Cases		**69**	**PASS**

C.2 Testing Execution Details

All tests are configured under the Maven Surefire configuration to run isolated inside the JVM. The test logs are redirected to clean local files inside the `log/` folder:

- **JVM Native Errors**: Redirected to `log/hs_err_pid*.log`.
- **Surefire Replay Logs**: Redirected to `log/replay_pid*.log`.
- **General System Output**: Redirected to `log/app.log`.

This ensures that the project root directory remains clean and uncluttered.